

Make or scale? Designing and optimizing a manufacturing system from launch to high volume



You designed a yo-yo in your garage and posted it on TikTok. In one week the video reached thousands of views, and you already hold 81 firm purchase intentions. It might be a flash in the pan, or it might be the start of a real business, you don't know yet. You decide to set up a first production process and start selling online. Because demand is small, uncertain, and customers want their yo-yo *personalized*, you need a process that makes highly customized, low-volume parts without betting a lot of money up front.

The yo-yo line

Model the line the way it will actually sit on the floor. Parts are built on a bank of 3D printers, which are the core of the operation and its constraint. What comes off those machines is not finished. It goes to post-processing, where the parts are cleaned and the support structures and other printed material that do not belong to the product are removed. From there the pieces move to assembly, where they are brought together into a working yo-yo, and finally to quality, where each unit is tested, inspected, and packed for shipment.

Demand forecast

For demand planning, we assume that the model and its analysis cover a period of one year. Accordingly, we define system operation as 240 working days per year at 8 hours of operation

per day. These parameters are fundamental: together they set the base production time available to us for delivering yo-yos to our customers.

The primary input is our expected annual yo-yo sales, the variable **NREF**. Because the actual arrival pattern is not known, we use an estimate: we model the time between arrivals with an exponential distribution. Because additive manufacturing printers benefit from batch production, the first step of the process must reflect this: incoming customer orders are grouped into batches that are then sent to the bank of 3D printers.

Entity Arrival Logic. The Source creates entities of type **YoYoBatch**, meaning each entity represents a full print batch (a build job with several yo-yo parts on the plate), not a single yo-yo. Arrival Mode is set to Interarrival Time, so arrivals are governed by a formula for the gap between consecutive batches rather than a fixed schedule or table.

Table 1. Input parameters for demand forecast.

Property	Setting	What it does	Why it matters
Entity Type	YoYoBatch	Each entity is one full build batch (NNEST yo-yos), not a single yo-yo	Fixes the unit of flow; parts are exploded downstream
Interarrival Time	Random.Exponential(Run.EndingTime*NNEST/NREF)	Inner term is the mean gap between batches, calibrated so releases sum to NREF yo-yos over the run	Models the customer orders arrivals

Maximum Arrivals	<code>Math.Ceiling(NREF/NNEST)</code>	Number of batches needed to cover annual volume	Stops the Source once the yearly order book is filled
-------------------------	---------------------------------------	---	---

Uses parameters: `NNEST`, `NREF`. Uses run reference: `Run.EndingTime`.

Additive manufacturing

The data needed to parameterize the printer bank is given in the tables below, and it comes in two forms: static properties, which are fixed data inputs that act as control variables, and functions, which are analytical expressions evaluated each time they are called.

A key decision is the number of printers (`NMAIN_AM`) needed to run the process, which depends on several variables, not all known with certainty.

Because a printer's work splits into subtasks, we model the build as a Task Sequence of two steps. The first is pre-processing, a variable time that prepares the printer for a build. The second is the build itself, the time to produce all the yo-yos in the batch. (Tip: Task Sequence is the natural way to represent this.)

Printers fail, which lowers their productive capacity. The Uptime Between Failures and Time To Repair distributions capture this and make the simulation more realistic than the deterministic cost model: breakdowns create queueing, so effective capacity falls below the theoretical maximum.

On cost, Capital Cost `FC_AM` carries the fixed investment in printers and space, annualized for comparison. Cost Per Use `VMAT_AM*NNEST` charges material once per batch, and Usage Cost Rate `PWR_AM*ELEC` charges energy while the printer runs.

Tip: you do not build one Server per printer. You build one Server, set its capacity to the number of printers, attach the two-task build sequence, and let the reliability and cost expressions do the rest. That is what turns a single object into a realistic printer bank.

Table 2. Input parameters for the printers.

Property	Enter this	What it does
----------	------------	--------------

Initial Capacity	<code>NMAIN_AM</code>	Sizes the whole AM bank;
Task 1: Pre-processing	Processing Time [min] = <code>Random.Uniform(5, 10)</code>	Variable prep time to set the printer up for a build
Task 2: Build	Processing Time [sec] = <code>Random.Triangular(0.95*TBUILD, TBUILD, 1.15*TBUILD)</code>	Time to print all <code>NNEST</code> yo-yos in the batch
Uptime Between Failures	<code>Random.Exponential(MTTF_AM)</code>	Random productive time to next breakdown
Time To Repair	<code>Random.Triangular(0.95*MTTR_AM, MTTR_AM, MTTR_AM*1.20)</code>	Random repair duration around the mean
Capital Cost	<code>FC_AM</code>	Annual fixed cost of printers + space
Cost Per Use	<code>VMAT_AM*NNEST</code>	Material charged once per batch
Usage Cost Rate	<code>PWR_AM*ELEC</code>	Energy charged while printing

Uses parameters: `NNEST`, `MTTF_AM`, `MTTR_AM`, `PWR_AM`, `ELEC`. Uses functions: `NMAIN_AM`, `TAM`, `VMAT_AM`, `FC_AM`.

Post-processing

After the building operation, each completed build passes through a post-processing step where the finishing operations are carried out: support and flash removal, cleaning, curing/handling, and a final visual check. Once finished, the parts move on to the assembly station. The key point is that the AM printer produces several yo-yos in a single build, and post-processing is carried out on the whole build at once, batch by batch.

Tip: check out the Separator object, which allows the batch to be broken apart into individual yo-yos for cleaning. A New Entity Type **YoYoBody** can be created to model exactly what comes out: one batch explodes into **NNEST** individual yo-yo bodies.

The Initial Capacity **POSTPROC_CAP** sizes the post-processing station, the number of parallel manual cleaning workstations; that is, each unit of capacity corresponds to one worker. **POSTPROC** applies a defined cleaning time per batch. In this production step, the relevant cost is labor, charged only while post-processing is actually running.

Property	Enter this	What it does
Initial Capacity	POSTPROC_CAP	Sizes the station; each unit of capacity = one worker
Processing Time	POSTPROC	Finishing time applied at this station
Usage Cost Rate	WAGE	Labor charged only while running

Uses parameters: **NNEST**, **POSTPROC_CAP**, **POSTPROC**, **WAGE**. Creates entity: **YoYoBody**.

Assembly station

From post-processing, the finished parts move to the assembly station, where the individual components are put together into a complete yo-yo (the two halves joined on the axle/bearing and the string fitted). Unlike post-processing, assembly is done one yo-yo at a time rather than a whole build at once.

The assembly station must be sized through its capacity: **ASSEMBLYCAP** sets the assembly area as a number of parallel manual stations, where each unit of capacity is one assembly worker, exactly as **POSTPROC_CAP** did for cleaning.

The Processing Time applies a per-unit assembly time. As with post-processing, labor is charged only while assembly is actually running.

Property	Enter this	What it does
Initial Capacity	ASSEMBLYCAP	Sizes the station; each unit of capacity = one worker
Processing Time	Random.Uniform(40, 60)	Per-unit assembly time, in sec.
Usage Cost Rate	WAGE	Labor charged only while running

Uses parameters: ASSEMBLYCAP, WAGE.

Quality check station

After assembly, each yo-yo goes to the drop test for quality check, where it is functionally tested with a drop/spin test and inspected against the quality criteria. This station is the gate that separates good units from rejects, so its outcome feeds both the good-unit count and the scrap logic.

The Capacity parameter QUALCAP sizes the QC area as a number of parallel manual inspection stations, where each unit of capacity is one inspector. As with assembly and post-processing, the entire cost lives in Usage Cost Rate WAGE: labor is charged only while a test is actually running.

Not every yo-yo passes. A certain share of production comes out defective, and the process has to account for that reality.

Tip: The important requirement here is that good and defective units cannot follow the same path. Once a yo-yo is tested, the process has to make a decision about it and then send it where it belongs: the units that pass carry on toward the customer as saleable product, and the units that fail are pulled out of the flow and scrapped.

Property	Enter this	What it does
Initial Capacity	QUALCAP	Sizes the station; each unit of capacity = one inspector
Processing Time	Random.Triangular(0.9, 1.9, 2.0)	Per-unit test time [min]
Usage Cost Rate	WAGE	Labor charged only while running

Uses parameters: QUALCAP, WAGE, FFAIL.

Challenge 1: create a low volume and high customization process

Situation. Demand is in the tens to low thousands, every unit may be different, and you can't justify expensive tooling for something that might not last.

Task. Pick a specific additive manufacturing process (FDM, SLS/MJF, MJ) and do a *detailed* design of the production process around it: workstations, cycle times, capacity and yield.

Tip: You may want to consider a hybrid strategy that combines different additive manufacturing technologies.

Use the Excel cost model as a source for the input parameters of your simulation, and as a benchmark to verify it.

The baseline scenario

Before you tune anything, you have to understand the business you are actually building. Run the line at your launch demand, with the smallest configuration that can meet it, and treat that as your day-one company: the shop as you would open it, sized to the orders you have realistically won, with your capital committed and your unit cost locked in. This is your reference point. Every decision you make later, about batch sizes, delivery promises, and when to scale, will be judged against this baseline, so it pays to understand exactly what is driving its numbers before you spend a euro changing them. With the baseline running, answer the following.

Trace where your startup capital goes. Show how your launch demand determines how many printers you have to buy on day one, and how that printer count then sets your upfront investment in equipment and facility space.

An initial model with all the required properties and functions is provided and should be used as your starting point. Your main goal is to use this base model to build out each step of the production process, working with the variables that are already defined.

The delivery scenario

In the baseline you sized the line to meet demand at a defensible cost. Now the pressure changes. Customers are no longer just buying yo-yos; they expect them quickly, and you have set yourself a hard promise: every order shipped in under twelve hours.

The optimization scenario

Define and run an optimization procedure in the simulation model, and use it to recommend the settings that best run your factory across both objectives, cost and delivery.